Contents

Demo: a relay broadcast program, checked for correctness, for extra work, and under a CPU cap **1**
Summary . . . 1
Words used in this document . . . 2
1. The program . . . 3
2. HydroFrac: can members disagree? . . . 6
3. Cress: does delay cause extra work? . . . 7
4. Sedge: the program under a CPU cap . . . 8
Setup . . . 8
Headline: one burst, two endings . . . 10
Round 3: how often it gets stuck (isolated cores, 5 seeds per setting) . . . 12
Round 2: large-margin results only . . . 14
Round 1: three policies across load levels . . . 14
Is the collapse kept going by the extra work? . . . 17
Time series . . . 19
CPU headroom at baseline . . . 21
5. Discussion . . . 21
Caveats . . . 26
Appendix A: Methods . . . 27
Isolated cores (round 3) . . . 27
Measuring capacity . . . 27
The silent-tail rule . . . 27
One outside-only classification for the fraction tables . . . 28
The lossy transport . . . 28
The counter-recording fix . . . 28
Appendix B: Commands . . . 28
Appendix C: Where the data is . . . 29

Demo: a relay broadcast program, checked for correctness, for extra work, and under a CPU cap

Summary

The program. One source sends each request to nine members. Each request is an id and a 4 KiB body. If the source crashes part-way through, some members have the request and others do not. So the members forward (“relay”) what they receive to each other. The policy studied here re-sends every relay until the receiving member acknowledges it. The program has no code that limits its own load. Its capacity comes only from the CPU cap that we impose from outside.

Three tools, three questions.

- **HydroFrac: can the members end up disagreeing?** It injects crashes and lost messages and searches for an end state in which two live members hold different sets of requests.
  - The simple policy, which forwards each request once, fails. One lost relay plus one source crash is enough.
  - The fix, which re-sends every relay until it is acknowledged, passes. But it passes only when it never gives up. With a limit of 3 or 5 sends, HydroFrac finds a failure.
- **Cress: does a delay make the program do extra work?** It runs the program in a simulator, holds back one internal step at a time, and counts the extra messages.
  - The fix is flagged HAZARDOUS. Late acknowledgements cause re-sends. Each re-send causes more sending, merging and acknowledging. Under the longest hold, the busiest member sends 175.60 times its normal count.
- **Sedge: what happens on real machines?** It runs each member in a container with a small CPU cap. It offers a fixed request rate with a short burst on top.

The headline result.

- Without a burst, the fix is stable at every load level tried. That covers 37 no-burst runs, all clear by the counters read from outside the program.
- After a burst, the members fall into a **re-send storm**: they re-send far more than usual. The storm then either dies down, or it ends in a **black hole**. In a black hole the members' CPU stays at the cap, they hand almost nothing to the kernel, and nothing is ever acknowledged again.
- Whether it gets stuck depends on the size of the burst. On isolated cores, at a load of one quarter of capacity, with 5 runs per burst size:
  - twice capacity for 1 s, and 1.5 times capacity for 1.5 s: 5 of 5 stuck;
  - 1.5 times capacity for 1 s: 4 of 5 stuck;
  - twice capacity for 0.5 s: 5 of 5 stuck after 60 s, and 3 of 5 still stuck after 180 s.
- After a burst of twice capacity for 5 s, it is stuck at every load level tried, from two thirds of capacity down to one eighth.
- One run shows both phases. First comes about 150 s of storm on the wire: kernel writes are 3.3 times normal, sends handed to the send queues are 13 times normal, and the members report about 150 thousand re-sends per second. Then comes the black hole: kernel writes fall to 2–3 % of normal, attempted sends are 31 times normal, and 99.9 % of them are dropped because the queue is full.
- A variant that gives up after 3 sends also storms after the same burst. But its storm always ends. It never got stuck within 180 s. How long the storm lasts grows steeply with the load.

Honest caveats.

- **In the black hole, the extra work is not on the wire.** Kernel writes fall to a few percent of normal. The extra work shows only as attempted sends, counted by the transport layer, and as CPU held at the cap. The storm before the black hole is where the extra traffic really is on the wire.
- **Capacity is uncertain.** Each capacity estimate is uncertain by about ±5–9 %, so every boundary given in units of capacity is a range.
- **One claim was dropped.** An earlier draft said the extra work was "necessary but not sufficient" for the collapse. This version does not make that claim.
- Further limits are listed under Caveats.

What changed since the previous version. The previous version reported that the per-peer policy collapsed at high load. That collapse was not caused by retries. It was caused by the program rebuilding its whole list of unacknowledged relays on every step. With that list kept instead of rebuilt, the per-peer policy recovers from the same burst at every load tried. Its re-sends are about 4 % of normal traffic.[1] This version therefore studies the per-entry policy, whose re-sends grow with the backlog, and asks whether that extra work keeps a collapse going.

Contents:

1. The program
2. HydroFrac: can members disagree?
3. Cress: does delay cause extra work?
4. Sedge: the program under a CPU cap
5. Discussion
6. Caveats

Appendices: A. Methods, B. Commands, C. Where the data is.

Words used in this document

- **Member, source.** The source sends requests. The nine members receive and relay them.
- **Relay.** One member forwarding one request to one other member.
- **Ack.** An acknowledgement: a short message saying "I got your relay".
- **Unacked relays, backlog.** The relays a member has sent that are not yet acknowledged.

[1] `relay_broadcast_amplification/README.md`, Part 1.

- **Step.** The program processes input in steps. In each step a member takes everything that has arrived since the last step and handles it as one batch.
- **Timer tick.** A step that includes a beat of the program's own 50 ms clock. Re-sends are decided only on timer ticks.
- **Round.** One request from the source. "Rounds per second" means requests per second.
- **μ (mu).** Capacity: the highest request rate the deployed system keeps up with. It is measured, not set.
- **λ (lambda).** The request rate actually offered.
- **f.** Headroom: f = μ / λ. At f = 2 the system is offered half of what it can handle. At f = 4, a quarter. A larger f means a lighter load.
- **Burst, or trigger.** A short period in which the offered rate is raised, for example to 2μ (twice capacity) for 0.5 s.
- **Tail.** The part of the run after the burst: 60 s or 180 s.
- **Baseline.** The rate of a counter before the burst, from 5 s to 10 s into the run. Ratios such as "3.3×" are against the run's own baseline.
- **Control.** A run with the same settings as a burst run, but no burst.
- **Open loop.** Requests arrive on a fixed schedule, whether or not the system keeps up. A slow system does not slow the arrivals down.
- **Throttled.** The kernel pauses a container that has used up its CPU allowance for the current period. It resumes at the next period.
- **In band.** The members' own once-a-second reports. They travel the same queues as the members' traffic, so a jammed member stops reporting.
- **Out of band.** Counters that Sedge reads from outside the program: kernel write counts, memory, CPU use, and the send-queue counters kept by the transport layer.
- **Kernel writes.** Bytes a member actually wrote to its sockets, as counted by the kernel.
- **Attempted sends.** Messages the program handed to its outgoing send queues. A send that finds its queue full is dropped and counted.
- **Storm.** Attempted sends stay above 1.5 times baseline.
- **Black hole.** Kernel writes fall below a small fraction of baseline while CPU stays at the cap. The members are busy but get nothing through.
- **Stuck.** In a storm or a black hole at the end of the run. **Clear** means neither.
- **Sedge's verdicts.** COLLAPSED, DRAINING (the backlog is falling), RECOVERED, and INVALID (the run cannot be judged, for example because a node crashed). The fraction tables also give a separate outside-only classification; see Appendix A.

1. The program

The program has one source and a cluster of members.[2]

- The **source** sends each request to every member.
- The **members relay** each request to their peers, so that a source crash does not leave some members without it. The relay policy is one of:
  - **None:** no relaying.
  - **Echo:** a member that gets a request *from the source* forwards it once to every peer.
  - **EchoResend:** Echo, plus each echo is re-sent until the peer acknowledges it.
  - **Per-peer (ReliableResend):** a member relays every request the first time it gets it *from anyone*. It keeps each relay on a list until the peer acknowledges it. On each timer tick it re-sends at most one overdue relay per peer.
  - **Per-entry (ReliableResendPerEntry):** the same relaying as per-peer. But on each timer tick it re-sends *every* relay whose ack is overdue, with no per-peer or per-tick limit. A relay is overdue once it has waited 3 timer ticks (150 ms) for its ack. An optional limit gives a relay up after a set number of sends. With no limit, it is never given up.

The correctness claim. Each member keeps the set of request ids it has received. The program

[2] `hydro_test/src/cluster/relay_broadcast.rs`. The policy is chosen by its `RelayPolicy` setting. The per-entry policy is `ReliableResendPerEntry { ack_timeout_ticks, max_attempts }`: `ack_timeout_ticks` is the ack wait in timer ticks, and `max_attempts` is the send limit (`None` means no limit).

claims that this set is *eventually consistent*: in the end, every live member holds the same set. The claim is backed by a hand-written argument, and HydroFrac's job is to check it.

The claim as written in the code is below.[3] The comment inside it is the hand-written argument. It also notes that None, Echo and EchoResend carry the same claim without the argument.

```
let delivered = initial
    .clone()
    .weaken_consistency()
    .map(q!(|r| r.id))
    .merge_unordered(relays_in.clone().map(q!(|r| r.id)))
    .assert_has_consistency_of::<Cluster<'a, Member, EventualConsistency>>(manual_proof!(
        /** reliable broadcast + resend-until-ack (`ReliableResend`): a member relays every
        id to all peers on its first receipt from any sender, source or peer, and re-sends
        each relay until the peer acknowledges. A live member that received id has
        therefore relayed it to every peer, and re-sends past any omission until acked, so
        every live member eventually receives every id that any live member received.
        `None`, `Echo` and `EchoResend` share this label but not the argument: they relay
        only ids received from the source, which HydroFrac refutes */
    ));
```

Why Echo is not enough. An echo is sent once. If it is lost, nothing repairs it. If the source also crashed before reaching that member, the member never learns the id.

The fix: per-entry re-sending. It adds two kinds of redundancy:

- A member relays an id the first time it hears it from anyone. So if a relaying member crashes part-way through, any peer it did reach passes the id on.
- Each relay has its own retry timer and is re-sent until acknowledged. So a lost relay is repaired, however many times it is lost.

The re-send loop. The excerpt below is what runs on a timer tick.[4] It walks the overdue relays in deadline order, re-sends each one, and either restarts its deadline or, at the send limit, gives it up.

```
TableEvent::Pump { now } => {
    let mut out = Vec::new();
    if ack_timeout_ticks > 0 && per_entry {
        // Every relay whose deadline has passed, in deadline order.
        while let Some(&(deadline, peer, id)) = self.due.first() {
            if deadline > now {
                break;
            }
            self.due.pop_first();
            let e = self
                .entries
                .get_mut(&(peer, id))
                .expect("every deadline has its entry");
            e.attempts += 1;
            out.push(TableOut::Wire {
                peer,
                id,
                body: e.body.clone(),
                resend: true,
            });
            if max_attempts > 0 && e.attempts >= max_attempts {
                // The last attempt: give the relay up.
                self.entries.remove(&(peer, id));
                self.len -= 1;
            } else {
                e.deadline = now + ack_timeout_ticks;
                self.due.insert((e.deadline, peer, id));
            }
        }
    }
    // ...
}
```

[3] `relay_broadcast.rs:571-583`. `assert_has_consistency_of` is the Hydro call that attaches a consistency claim to a collection; `manual_proof!` holds the hand-written argument.

[4] `relay_broadcast.rs:933-961`. `Pump` is the name of the timer-tick event in the program's list of unacked relays.

- On each timer tick, the number of re-sends equals the number of overdue relays.
- When acks are late across the board, every relay is overdue. So the re-send load grows with the backlog.
- The per-peer policy, by contrast, re-sends at most one relay per peer per tick.

The list of unacked relays is kept, not rebuilt. Each member keeps a list of the messages it has forwarded but not yet had acknowledged. The list lasts for the whole run and is updated one piece at a time: an acknowledgement removes one entry, a new message adds one per peer, and a timer tick looks only at overdue entries. The same piece of state also remembers which ids the member has already seen, so it relays each id only once. The old version rebuilt the whole list on every step, and that, not the retries, caused the previous demo's collapse.[5]

The Sedge settings. The block below tells Sedge how to deploy the program: nine members, the per-entry policy with no send limit, a starting guess for capacity, and the CPU cap.[6]

```
#[cfg_attr(feature = "sedge", sedge::overload_check(
    name = per_entry_n9_4k,
    members = 9,
    config = RelayConfig { policy: RelayPolicy::ReliableResendPerEntry { ack_timeout_ticks: 3, max_attempts: None },
↪   channel: RelayChannel::FailStop },
    mu = 150,
    cpus(Member = 0.01),
    cpu_period_us = 100_000,
    timers(timer = Duration::from_millis(50), report_tick = Duration::from_secs(1)),
    round_data = |round, inputs| {
        inputs.requests.send(Request::with_body(round as u64, 4096));
    },
))]
```

- `mu = 150` is only where the capacity probe starts. Capacity is measured.
- The default cap in this block is 1 ms of CPU in every 100 ms. **All runs below override it** to 1 ms in every 20 ms (0.05 of a core). Under the old cap a paused member stayed paused for 80–95 ms at a time. That is longer than the 50 ms timer and the 150 ms ack wait, so every pause looked like a lost ack. Under the new cap a pause lasts under 20 ms (§4).
- The source has no cap and is fed open loop.
- Both clocks run inside the program: the 50 ms re-send clock and the once-a-second report.

No capacity code. A member merges whatever has arrived. Its capacity is whatever the code achieves on 5 % of a core. There are no per-step budgets, no filler load, and no clocks driven by the test harness.

Program changes since the previous version. HydroFrac, Cress and Sedge were re-run after each one.

1. **The list of unacked relays is kept, not rebuilt.** This removed the per-step cost that grew with the backlog, which kept the previous version's collapse going (§5).[7]
2. **Duplicates are detected by id alone, inside that same kept state.** The old check compared id and body, which kept every received body in memory. That caused a slow memory creep of about 0.3 MB/s even without a burst.
3. **The per-entry policy**, described above.

[5] In Hydro terms the list is one top-level `scan` (a fold that keeps its state across the whole run) over a stream of `TableEvents`: `Ack`, `Receipt` (first receipt) and `Pump` (timer tick). An ack is one `remove`, a first receipt is one `insert` per peer, and a timer tick costs work in proportion to the overdue entries. The duplicate check is a `HashSet` of ids in the same `scan`. The `scan` is at `relay_broadcast.rs:672-673` and `TableEvent` at `:802`. The members' four decision points (`use::batch`: timer, arrivals, first receipts, acks) are at `:615-618`.

[6] `relay_broadcast.rs:487-498`. The runs override the cap on the command line with `--cpus Member=0.05 --cpu-period-us 20000`.

[7] The old list was a tick-scoped `KeyedSingleton`: Hydro state that lives for one step and is rebuilt on the next. The old duplicate check was a top-level `unique` over `(id, body)`.

2. HydroFrac: can members disagree?

HydroFrac searches for a set of faults (crashes and lost messages) under which members end up holding different sets of ids. If it finds one, it reports a **counterexample**. If its search runs out without one, it reports "no violation found". That is not a proof that none exists.

The test setup:[8]

- **Size:** 3 members and 2 request ids. The source and every member can crash.
- **Channel:** any single message may be lost.
- **Two schedules for the timer.**
  - *Standard:* the test sends both requests and waits for quiet. Then it declares the end of faults (the *cut*: no new crash or loss after it), and then it fires the timers. So every retry after the cut is delivered.
  - *Stricter:* 3 × (send limit) + 1 timer rounds fire *before* the cut. So HydroFrac can drop the retries as well as the first sends.
- **Sampling:** for each set of faults, 8 message orderings are sampled.

The log below is from a re-run of 16 tests, all passing: 12 HydroFrac tests and 4 unit tests of the unacked list.[9] Each test prints its verdict just before its own name line, and "…" marks lines cut from the log.

```
$ cargo test -j 6 -p hydro_test --lib cluster::relay_broadcast -- --nocapture --test-threads=3
  delivered  :576                   DIVERGES under omit(relay Relay { from: 0, id: 2, body: Body(b"") } member
  ↪  0→member 1) + crash(source after sending bcast Request { id: 2, body: Body(b"") } to member 0)
relay_broadcast Echo over LossyDelayedForever, 0 timer rounds before the cut: 2.8s
...
  delivered  :576                   DIVERGES under omit(relay Relay { from: 0, id: 2, body: Body(b"") } member
  ↪  0→member 1) + omit(relay Relay { from: 0, id: 2, body: Body(b"") } member 0→member 2) + crash(source after
  ↪  sending bcast Request { id: 2, body: Body(b"") } to member 0)
relay_broadcast ReliableResendPerEntry { ack_timeout_ticks: 3, max_attempts: Some(3) } over LossyDelayedForever, 10
↪  timer rounds before the cut: 12.0s
...
  delivered  :576                   DIVERGES under omit(relay Relay { from: 0, id: 2, body: Body(b"") } member
  ↪  0→member 1) + omit(relay Relay { from: 0, id: 2, body: Body(b"") } member 0→member 2) + crash(source after
  ↪  sending bcast Request { id: 2, body: Body(b"") } to member 0)
relay_broadcast ReliableResendPerEntry { ack_timeout_ticks: 3, max_attempts: Some(5) } over LossyDelayedForever, 16
↪  timer rounds before the cut: 15.3s
...
  delivered  :576                   no violation found (249 fault sets, 8 sampled schedules each)
relay_broadcast ReliableResendPerEntry { ack_timeout_ticks: 3, max_attempts: None } over LossyDelayedForever, 0
↪  timer rounds before the cut: 167.5s
...
  delivered  :576                   no violation found (243 fault sets, 8 sampled schedules each)
relay_broadcast ReliableResendPerEntry { ack_timeout_ticks: 3, max_attempts: None } over LossyDelayedForever, 10
↪  timer rounds before the cut: 197.3s
...
  delivered  :576                   no violation found (311 fault sets, 8 sampled schedules each)
relay_broadcast ReliableResend { ack_timeout_ticks: 3 } over LossyDelayedForever, 0 timer rounds before the cut:
↪  226.6s
...
test result: ok. 16 passed; 0 failed; 0 ignored; 0 measured; 240 filtered out; finished in 509.31s
```

policy	timer rounds before the cut	result
Echo	0	counterexample: relay of id 2 from member 0 to member 1 lost, plus a source crash
per-entry, limit 3 / 5 / none, any number of crashes	0	no violation (253 / 248 / 249 fault sets)
per-entry, limit 3	10	counterexample
per-entry, limit 5	16	counterexample
per-entry, no limit	10	no violation (243 fault sets)

[8] `hydro_test/src/cluster/relay_broadcast_ldfi.rs`. The channel is `lossy_delayed_forever`, and the sampling is `Exploration::Fuzz { instances: 8 }`. The stricter setup's count is `3 · max_attempts + 1`.

[9] `demo_relay_broadcast/hydrofrac.txt`, re-run on commit `52597a61df`.

policy	timer rounds before the cut	result
per-peer, any number of crashes / 1 crash	0	no violation (311 / 160 fault sets)

The number of fault sets differs by a few between runs, because the orderings are sampled. For example, the previous run found 247 and 239 for the two no-limit rows. The verdicts were the same in every run.

What this shows.

- **Echo fails with two faults.** The source sends id 2 to member 0 only, then crashes. Member 0 echoes id 2 to its peers, but the echo to member 1 is lost. Echo never re-sends. So member 1 never gets id 2, while members 0 and 2 have it.
- **A send limit is a hole.** In the stricter setup the source again crashes after reaching only member 0. Every send of member 0's relays of id 2, to members 1 and 2, is lost. Once member 0 reaches its limit, it gives the relays up. Re-sends are identical bytes, so a single "drop this message" fault covers every copy.
- **Only "never give up" passes.** The no-limit policy holds in both setups. It is the one deployed below. "No violation found" covers the sampled orderings at 3 members and 2 ids.
- **The redundancy is the hazard.** HydroFrac's case for the fix is exactly "keep re-sending until acknowledged". §3 shows that this is what Cress flags.

3. Cress: does delay cause extra work?

Cress runs the program in a simulator. It picks one internal decision point at a time (a place where the program takes whatever has arrived as a batch) and holds it back for a range of lengths.[10] It then counts every output and every message, and compares with a run that held nothing back.

The command and its one-line verdict are below. "Grows" means the extra work kept growing with the length of the hold.

```
./scripts/cress --config per_entry_n9_4k hydro_test relay_broadcast
```

```
! HAZARDOUS  relay_broadcast (per_entry_n9_4k)  growth: grows  multiplier: 175.60x  location:
↪  hydro_test/src/cluster/relay_broadcast.rs:616, batch of Arrival
    outputs: amplified: wire, merged, acks_sent, delivered; shed: outstanding_depth, metrics
```

Two decision points react: the members' batch of arriving relays (line 616) and their batch of arriving acks (line 618). The excerpt below is for the acks. Each column is a hold length; each row counts extra work against the run with no hold.[11]

```
hold length (rounds)                      1      2      4      8     16     32     64    128    199
hydro_test/src/cluster/relay_broadcast.rs:618, batch of Ack
    extra records admitted                0    432   2592  12096  42768 104976 229392 478224 627912
    extra messages, busiest sender        0     16     96    448   1584   3888   8496  17712  27936
    extra messages, all senders           0    144    864   4032  14256  34992  76464 159408 251424
    first reacted at a hold of 2 rounds with 432 extra; largest extra work 627912 in admitted records
    beyond the input (holds 16, 32, 64, 128): extra sends from Cluster(loc2v1) @8 keeps growing: rate 144.00 per
    ↪  round of delay, no ceiling within the run (rates 144.000, 144.000, 144.000)
    beyond the input (holds 16, 32, 64, 128): extra admitted records keeps growing: rate 3888.00 per round of delay,
    ↪  no ceiling within the run (rates 3888.000, 3888.000, 3888.000)
    multiplier: 175.60× the unheld run's 160 sends from Cluster(loc2v1) @8 (+27936 at a hold of 199 rounds)
    under a hold that never ends: +627912 admitted records (3155.3 per round of the run)
```

What this shows.

- **Late acks cause re-sends.** Holding back a member's acks makes every relay it sent look overdue. So it re-sends all of them every three timer ticks.
- **Late arrivals do the same to the peers.** Holding back a member's arriving relays means it does not ack them, so its peers re-send.

[10] The decision points are Hydro's `use::batch` and `use::snapshot` calls.

[11] `relay_broadcast_amplification/cress/per_entry_n9_4k.txt`.

- **Four outputs grow:**
  - the re-sends themselves (“wire”);
  - the relays received and merged again (“merged” and “delivered”);
  - the acks sent again for each merged duplicate (“acks_sent”).

  §4 measures the same four in deployment.
- **It grows with the backlog.** The busiest member gains 144 extra sends per round of delay, with no ceiling within the run. The per-peer policy gains 16, at a 25.95× multiplier, because it re-sends at most one relay per peer per timer tick.
- **The workload was reduced** to 10 rounds, a run length of 200, and *empty* bodies. With 40 rounds, a length of 1000 and 4 KiB bodies, the simulator keeps every buffered re-send in memory; it grew to 10.5 GB and was killed. Cress counts messages, not bytes, so body size does not change its counts. The Sedge runs keep 4 KiB bodies.

The other two policies deployed in §4. Cress was re-run on the final code for all three deployed configurations. Their verdict lines are below.[12]

```
! HAZARDOUS  relay_broadcast (per_entry_n9_4k)  growth: grows  multiplier: 175.60x  location:
↪  hydro_test/src/cluster/relay_broadcast.rs:616, batch of Arrival
! HAZARDOUS  relay_broadcast (per_entry3_n9_4k)  growth: stops  multiplier: 2.80x  location:
↪  hydro_test/src/cluster/relay_broadcast.rs:616, batch of Arrival
! HAZARDOUS  relay_broadcast (reliable_resend_n9_4k)  growth: grows  multiplier: 25.95x  location:
↪  hydro_test/src/cluster/relay_broadcast.rs:616, batch of Arrival
```

- **Per-entry with a limit of 3** (`per_entry3_n9_4k`) is HAZARDOUS, but its growth **stops** at 2.80×. A relay is sent at most three times, so no hold can cost more than about 3 times the sends.
- **Per-peer** (`reliable_resend_n9_4k`) is HAZARDOUS and grows, at a 25.95× multiplier. But it grows by 16 sends per round of delay, not 144.
- The two per-entry configurations use the reduced workload above. The per-peer one runs 40 rounds, a length of 1000 and 4 KiB bodies.

Cress models delay, not loss, and that is enough here.

- The Sedge runs give the members *lossy* send queues: a send that finds the queue full is dropped. Cress never drops a message; it only holds one back.
- Under a retry policy, a dropped relay followed by its re-send looks, to the rest of the program, exactly like one relay that was delayed until the re-send.
- The same holds for a dropped ack: the relay is re-sent until a later ack gets through.
- So every run of the lossy deployment is also a run of Cress’s delay model, and Cress’s verdict carries over. The lossy queue is a deployment option, not a program change, and needs no Cress check of its own.

4. Sedge: the program under a CPU cap

Setup

Three configurations. Each has 9 members, 4 KiB bodies, and a 3-tick (150 ms) ack wait.

- `per_entry_n9_4k`: per-entry, no send limit. This is the subject. Below: “per-entry”.
- `per_entry3_n9_4k`: per-entry, limit of 3 sends, which HydroFrac refutes. Below: “per-entry, limit 3”.
- `reliable_resend_n9_4k`: per-peer, at most one re-send per peer per timer tick. Below: “per-peer”.

CPU cap: 0.05 of a core, measured over 20 ms periods, for each member in its own container.

- The default cap (0.01 of a core per 100 ms) caused pauses of 80–95 ms. That is longer than the 50 ms timer and the 150 ms ack wait, so one pause could make every unacked relay overdue.
- Under the new cap a member at baseline pauses for 7–14 ms at a time. A member that is throttled all the time pauses for about 20 ms.[13]

[12] relay_broadcast_lossy/cress/console.txt. The full reports for all three are in relay_broadcast_lossy/cress/.

[13] Pause length is `throttled_ms / nr_throttled` in the traces.

- The source is uncapped and open loop.

Build and cores. A release build. The test harness and the program run on separate CPUs. Round 1 put the program on CPUs 7–8. Round 3 used isolated core sets (Appendix A).

Memory limit. 400 MB per member, 2000 MB for the source. If any node is killed for running out of memory, or exits, the run is INVALID. Sedge checks every node before teardown. A crash is not a collapse.

Counters read from outside the program. None of these depends on the members' own reports.

- **Kernel writes and memory**, read four times a second from a separate container. Kernel bytes written divided by 4120 bytes gives the number of relays a member actually wrote to its sockets.
- **CPU use and throttling**, from the containers' CPU accounting.
- **Attempted and dropped sends**, counted by the transport layer, outside the program's own logic. They are the sends handed to the send queue, and the ones the queue dropped because it was full.
  - Each member writes these counts to a file five times a second, and Sedge reads them once a second.
  - A starved member writes late, so one second can read 0 and the next carries two seconds' worth.
  - Averages over a window are exact. The plots smooth these two series over 3 s.

Send queues: lossy, 1024 messages. Each member's outgoing connection holds at most 1024 messages. A send that finds the queue full is dropped and counted. The source's feed stays open loop and unbounded. The other two options failed:

- **Unbounded queues** (the default).[14] A burst of twice capacity for 5 s fills each member's queues with first relays while its writers starve. A member is killed at 400 MB, at every f from 1.5 to 4. Those runs are INVALID: a crash, not a collapse. The memory comes from the burst itself, not from re-sends.
- **Bounded queues that push back on the member's own program.**[15] Below capacity this behaves like the unbounded case. Above capacity the members deadlock at 0 % CPU and 0 bytes written: each is blocked sending to a peer that is itself blocked sending, so no one reads. The source's queue then grows until the source is killed for running out of memory. No work is done, so there is no extra work to see.

A dropped send is a lost message. That is the fault HydroFrac already checked (§2). With retries, it is also a delay in Cress's model (§3).

Open loop held.

- In all 147 runs of rounds 1 and 3, no request entered the program more than 50 ms late. The worst lag was 48.4 ms.
- Of the 32 round-2 runs used here, one fails this check and is INVALID (see Round 2).

Capacity.

- Rounds 1 and 2 used μ = 565.7 rounds per second, measured for the per-entry configuration with the program on CPUs 3–4. At 565.7/s it kept up. At 616.9/s it fell behind, with unacked relays rising.[16]
- Other pairs of cores gave other values.
- Round 3 measured μ separately on each of its core sets (Appendix A).

Burst.

- Rounds 1 and 2: twice capacity for 5 s, starting 10 s into the run, then a 60 s tail.
- Sedge judges the second half of the tail, from 45 s to 75 s.
- Round 3 also varies the burst (1.5 or 2 times capacity, for 0.5–1.5 s) and the tail (60 or 180 s).

[14] `relay_broadcast_lossy/earlier/n_*`, same cap.

[15] The `bounded:Member` option: `earlier/e1_bounded_ctl_f2` and `earlier/bounded_member_calibration_old_cap.txt`.

[16] `relay_broadcast_lossy/probes.tsv` and `calibration.tsv`. Passed to Sedge with `--mu`. Other core pairs: `calibration_per_slot/` and `calibration_index.tsv`.

Controls. Each burst run has a control with the same f and length. In the control, the “burst” period runs at the normal rate, so the load never changes.

Headline: one burst, two endings

The setting.

- Per-entry, at f = 4, on core set S2 (μ = 713.5, λ = 178.4).
- A burst of twice capacity lasting only 0.5 s, then a 180 s tail.
- Each burst run is drawn with the control that has the same seed and length.[17]
- Every ratio is against the run’s own baseline (5–10 s).
- The phases are found from outside counters alone. A storm is attempted sends above 1.5 times baseline. A black hole is kernel writes below 0.1 times baseline.[18]

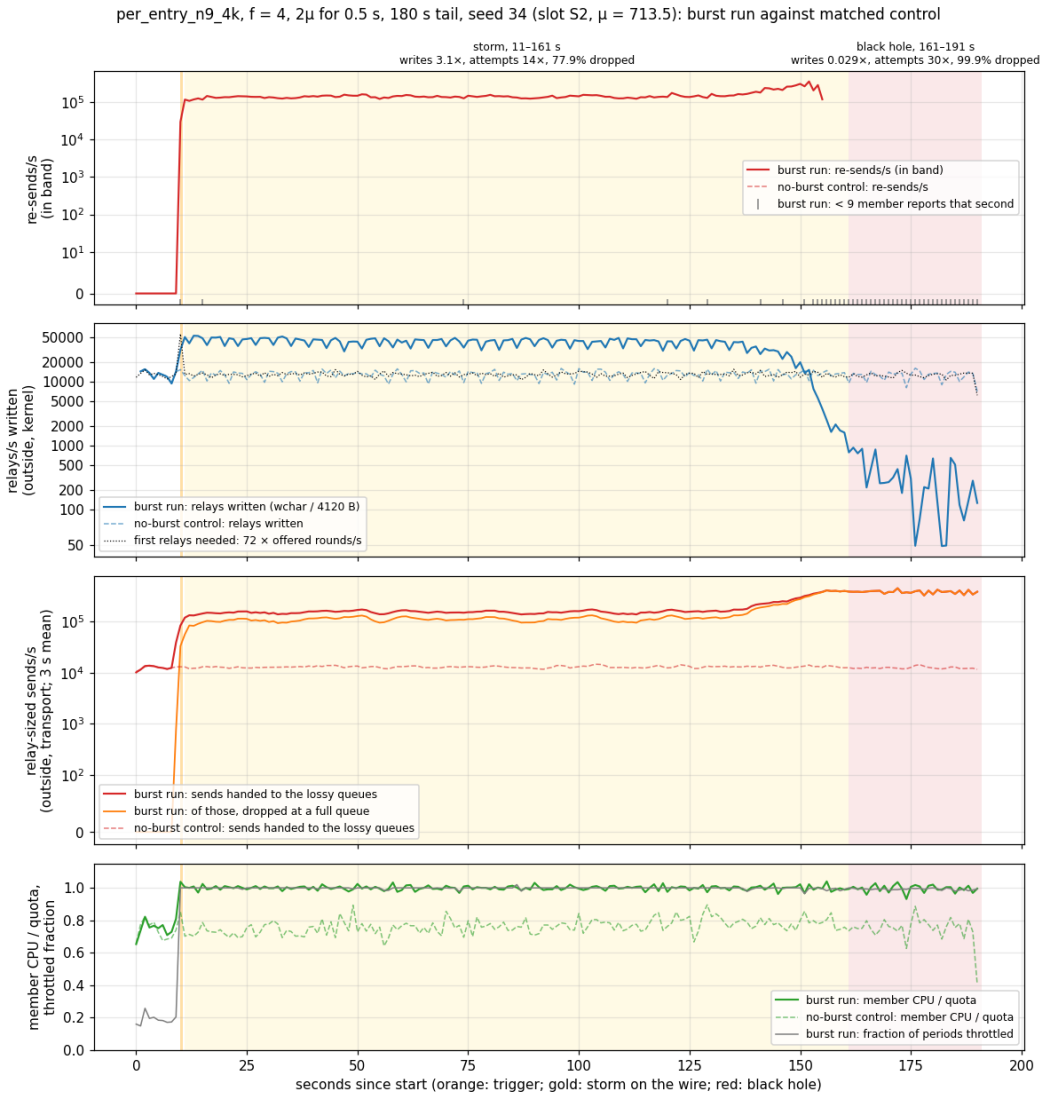


Figure 1: seed 34: storm on the wire, then the black hole

Seed 34.

- **Storm on the wire, 11–161 s.**

[17] Groups `r3_S2_pe_f4_m2_d0.5_t180_s5` (burst) and `r3_S2_pe_f4_ctl_d0.5_t180_s5` (control).
[18] `plot_headline.py`.

  - The members report about 152 thousand re-sends per second, and their reports keep arriving.
  - From 20 to 150 s, they write **3.3 times** their baseline relays to the kernel.
  - They hand their send queues 13 times baseline, and 75 % of that is dropped.
  - CPU is at the cap.
  - This is the extra work Cress predicted, on the wire.
- **Black hole, 161–191 s.**
  - From 165 to 190 s, kernel writes are **2.4 %** of baseline.
  - Attempted sends are **31 times** baseline, and **99.9 %** of them are dropped at a full queue.
  - The members' reports stop. CPU stays at the cap.
- **Verdicts.** Sedge says COLLAPSED ("resent at 106 489/s … unacked still rising"). The control with the same seed is clear throughout.

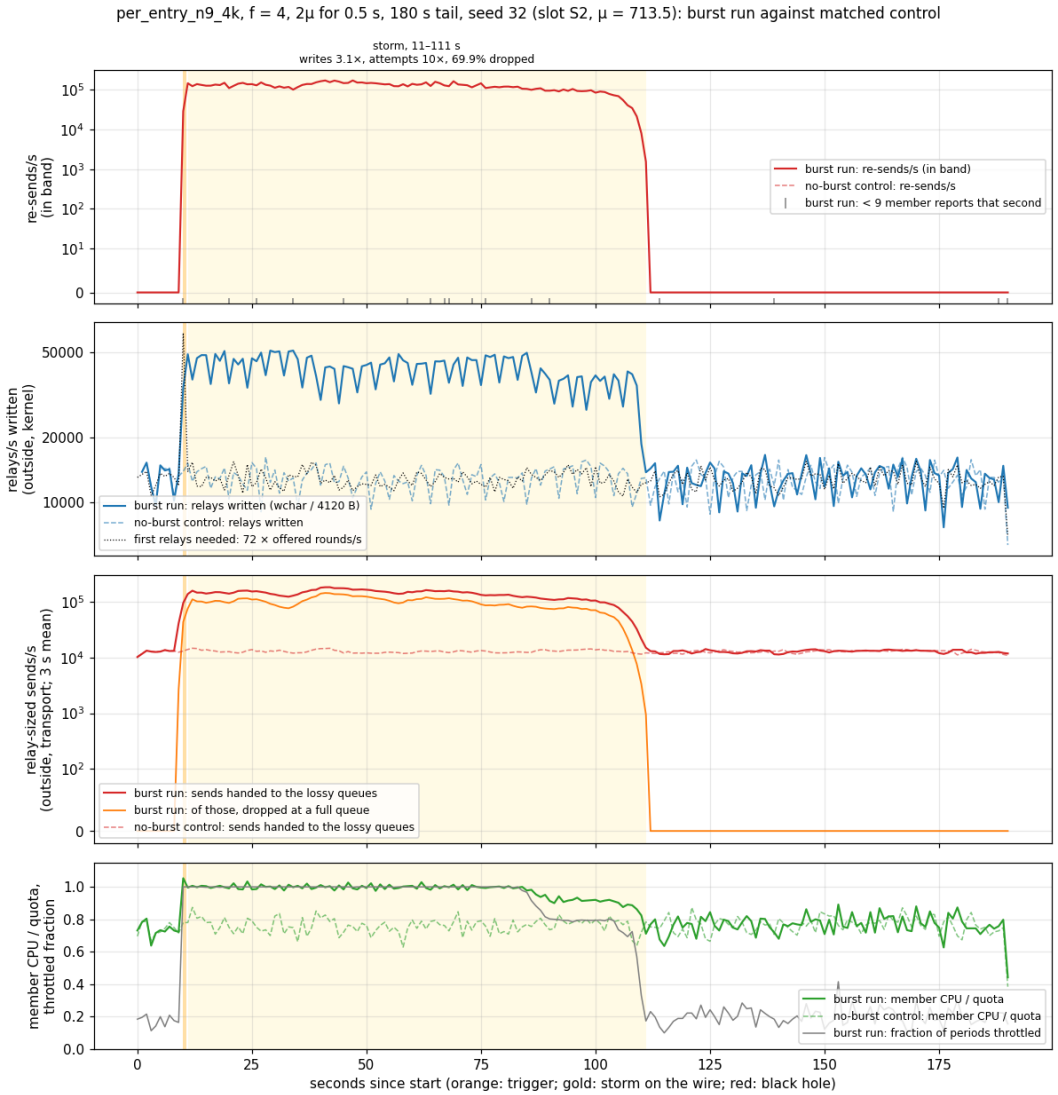


Figure 2: seed 32: the same burst, a storm that drains

Seed 32.

- Same burst, and the same kind of storm from 11 to 111 s: kernel writes 3.1 times baseline, attempted sends 10 times, 70 % dropped.
- Then it drains. Re-sends fall to 0. Attempted sends and kernel writes return to the control's level. CPU falls back to the control's 0.75 of the cap.

- Sedge says DRAINING, with its unacked relays still falling.

So the black hole is not a direct effect of the burst. It is a state that the storm can fall into: after more than two minutes of storm in seed 34, and after about 52 s in seed 31.

Round 3: how often it gets stuck (isolated cores, 5 seeds per setting)

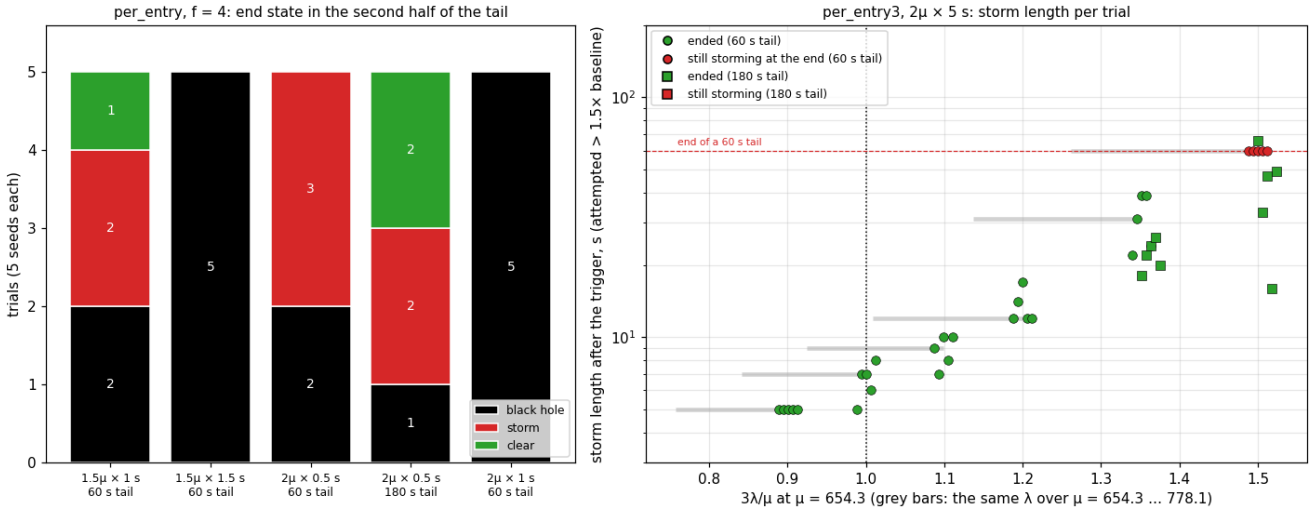


Figure 3: round 3: per-entry end states by burst, and per-entry-limit-3 storm length against 3λ/μ

Per-entry, f = 4, core set S2 (μ = 713.5). End states use the outside-only classification (Appendix A). The first table counts end states. The second gives Sedge's own verdicts for the same runs.

burst	tail	runs	black hole	storm	clear	stuck
2μ for 0.5 s	60 s	5	2	3	0	5 of 5
2μ for 0.5 s	180 s	5	1	2	2	3 of 5
1.5μ for 1 s	60 s	5	2	2	1	4 of 5
1.5μ for 1.5 s	60 s	5	5	0	0	5 of 5
2μ for 1 s	60 s	5	5	0	0	5 of 5
no burst (4 groups)	60 or 180 s	20	0	0	20	0 of 20

burst	tail	Sedge's verdicts
2μ for 0.5 s	60 s	4 COLLAPSED, 1 INVALID
2μ for 0.5 s	180 s	2 COLLAPSED, 1 INVALID, 2 DRAINING
1.5μ for 1 s	60 s	4 COLLAPSED, 1 RECOVERED
1.5μ for 1.5 s	60 s	5 COLLAPSED
2μ for 1 s	60 s	5 COLLAPSED
no burst (4 groups)	60 or 180 s	11 RECOVERED, 9 DRAINING

- **A larger burst gets stuck more often.** Bursts of 1.5μ for 1.5 s, or 2μ for 1 s, end in a black hole in 5 of 5 runs within 60 s.
- **The smallest burst does not decide the outcome by itself.** 2μ for 0.5 s leaves every run stuck at 60 s: two already in the black hole, three still storming. After 180 s:
  - two seeds (32, 33) have drained;
  - one (31) fell into the black hole at about 52 s;
  - two (34, 35) stormed until 140–157 s, and then fell in.
- **Neither label is a measuring artefact.** The black holes send no reports and are read entirely from outside. The storms have both their reports and their kernel writes.

Per-entry, limit 3, burst of 2μ for 5 s, core set S1 (μ = 654.3).

- 3λ/μ is the re-send load relative to capacity if every relay is sent three times, the most this policy allows.
- The storm length is the time after the burst during which attempted sends stay above 1.5 times baseline (3 s average).
- The first table gives the load. Because μ is uncertain, 3λ/μ is shown for the μ used and for the highest μ measured on this machine. The second table gives the outcomes.

f	3λ/μ at μ = 654.3	3λ/μ at μ = 778.1
2	1.50	1.26
2.22	1.35	1.14
2.5	1.20	1.01
2.73	1.10	0.92
3	1.00	0.84
3.33	0.90	0.76

f	tail	runs	still storming at the end	storm length (s)	Sedge's verdicts
2	60 s	5	5	60 or more (all)	2 COLLAPSED, 3 DRAINING
2	180 s	5	0	16, 33, 47, 49, 66	4 RECOVERED, 1 DRAINING
2.22	60 s	5	0	22, 24, 31, 39, 39	5 DRAINING
2.22	180 s	5	0	18, 20, 22, 24, 26	1 RECOVERED, 4 DRAINING
2.5	60 s	5	0	12, 12, 12, 14, 17	4 RECOVERED, 1 DRAINING
2.73	60 s	5	0	7, 8, 9, 10, 10	3 RECOVERED, 2 DRAINING
3	60 s	5	0	5, 6, 7, 7, 8	4 RECOVERED, 1 DRAINING
3.33	60 s	5	0	5 (all)	2 RECOVERED, 3 DRAINING

- **With a limit of 3, the storm always drained.** It never got stuck within 180 s, and no run reached a black hole.
  - At f = 2, a 60 s tail is too short to see the end.
  - In each of the five 180 s tails at f = 2 (seeds 31–35), the storm ended 16–66 s after the burst. While it lasted, re-sends ran at 36–43 thousand per second and kernel writes at 2.1–2.2 times baseline. Afterwards, writes were 0.95–1.01 times baseline.
  - At f = 2.22, the five 180 s storms ended after 18–26 s, with writes at 2.3–2.5 times baseline while they lasted. The tails were back at baseline (0.95–1.01 times).
- **The DRAINING labels on these runs are flat tails.** The 180 s runs are labelled 4 RECOVERED and 1 DRAINING at f = 2, and 1 RECOVERED and 4 DRAINING at f = 2.22. Every one is clear by the outside counters, and so are all 40 controls on S1.
- **Storm length grows steeply with 3λ/μ.** After the burst the storm lasts:
  - 5 s at 0.9 (just the burst's own backlog);
  - 5–8 s at 1.0;
  - 7–10 s at 1.1;
  - 12–17 s at 1.2;
  - 18–39 s at 1.35;
  - 16 s to more than 60 s at 1.5 (all five 60 s tails end while still storming).
- **Where the storm starts to outlast the burst.** Given the uncertainty in μ (654–778), this happens somewhere between 3λ/μ ≈ 1.0 and 1.35.
- **f = 2.22 at the end of its 60 s tail.**
  - Every storm had ended by 37–54 s. Attempted sends were back under 1.5 times baseline, and reported re-sends reached 0 by 40–57 s.
  - From 45 to 65 s, the five runs average 2, 45, 1 529, 11 261 and 14 667 re-sends per second, and 0.92–1.59 times baseline writes. That window straddles the end of the storm for three of them.

- From 65 to 75 s, they average 8–21 re-sends per second and 0.91–1.04 times baseline writes.

Round 2: large-margin results only

Round 2 shared physical cores between its core sets (Appendix A), so only results far from any boundary are used.

configuration	f	burst, tail	runs	end state (outside)	Sedge's verdicts
per-entry	2, 4, 8	2µ for 5 s, 60 s	3 each	black hole 9 of 9	9 COLLAPSED
per-entry, no burst	2, 4, 8	60 s	3 each	clear 9 of 9	3 RECOVERED, 6 DRAINING
per-entry, no burst	2, 4	300 s	1 each	clear 2 of 2	2 RECOVERED
per-peer	1.5, 2, 4	2µ for 5 s, 180 s	1, 4, 1	clear 6 of 6	5 RECOVERED, 1 INVALID (note 1)
per-peer, no burst	1.5, 2, 4	180 s	1, 4, 1	clear 6 of 6	3 RECOVERED, 3 DRAINING

1. Not open loop: 2 of 59 534 requests entered more than 50 ms late (worst 53 ms). The program's own reading was RECOVERED.

- **The black holes look the same as in round 1.** Kernel writes are 0.9–4.6 % of baseline. Attempted sends are 18 times baseline at f = 2, 37–39 times at f = 4, and 72–77 times at f = 8. 99.9 % are dropped.
- **Per-peer recovers given time.** Round 1 left it DRAINING after a 60 s tail. With a 180 s tail it recovers, at every f tried.

Round 1: three policies across load levels

One run per setting, one machine, cores not isolated.

- Tail rates are averages from 45 to 75 s, summed over the 9 members.
- "×" is against the same run's baseline (5–10 s).
- "Attempted" is the relay-sized sends the members handed to their send queues. "Written" is the relays they wrote to the kernel. Both are read from outside.

Per-entry, no limit:

f	λ	burst run	control	tail attempted/s	tail written/s	dropped
1.5	377.1	COLLAPSED	DRAINING (note 1)	348 k (12.7×)	173 (0.006×)	100.0 %
2	282.9	COLLAPSED	DRAINING (note 1)	371 k (16.7×)	185 (0.008×)	99.9 %
3	188.6	COLLAPSED	RECOVERED	373 k (27.1×)	188 (0.014×)	100.0 %
4	141.4	COLLAPSED	RECOVERED	386 k (40.0×)	225 (0.022×)	99.9 %
6	94.3	COLLAPSED	RECOVERED	384 k (58.0×)	255 (0.038×)	99.9 %
8	70.7	COLLAPSED	RECOVERED	395 k (79.3×)	247 (0.050×)	99.9 %

- In every collapsed run, member CPU is 0.99–1.00 of the cap, with 99 % of periods throttled.
- Member memory at 75 s is 95–98 MB.
- The uncapped source's memory at 75 s is 722, 529, 323, 221, 120 and 98 MB at f = 1.5, 2, 3, 4, 6 and 8 (see Caveats).

Per-entry, limit 3:

f	λ	burst run	control	tail attempted/s	tail written/s	dropped
2	282.9	DRAINING (note 2)	RECOVERED	58.8 k (2.80×)	47.8 k (2.25×)	19.5 %
3	188.6	RECOVERED	RECOVERED	13.5 k (0.99×)	13.6 k (0.98×)	0
4	141.4	RECOVERED	RECOVERED	10.3 k (1.14×)	10.3 k (1.04×)	0

Member CPU against the cap, fraction of periods throttled, and memory at 75 s:

- f = 2: 1.00, 0.99, 47 MB;
- f = 3: 0.87, 0.32, 45 MB;
- f = 4: 0.72, 0.17, 46 MB.

Per-peer:

f	λ	burst run	control	tail attempted/s	tail written/s	dropped
2	282.9	DRAINING (note 3)	DRAINING (note 1)	21.7 k (1.05×)	21.8 k (1.03×)	0
4	141.4	DRAINING (note 3)	DRAINING (note 1)	11.6 k (1.26×)	11.7 k (1.17×)	0

Member CPU against the cap, fraction of periods throttled, and memory at 75 s:

- f = 2: 0.98, 0.77, 55 MB;
- f = 4: 0.84, 0.26, 56 MB.

Notes:

1. **Not a real drain.** Sedge's "unacked rising" is noise in the slope of a flat line. Through the whole run, the control's unacked relays stay at 50–400. Outside writes are 0.95–1.02 times the 72 first relays per request, and nothing is dropped. These controls are stable.
2. **A stable but degraded state.**
   - Unacked relays fall slowly, from about 6.4 thousand at 20–30 s to 5.2 thousand at 75 s. Sedge reads this as falling at 23 per second.
   - Re-sends hold at 38.5 thousand per second against 20.2 thousand first relays per second. So nearly every relay is sent three times, the most the limit allows.
   - Members stay at the cap, but their reports keep arriving (9 per second) and every request is delivered.
   - Round 3 shows that this storm ends within 16–66 s at f = 2, given a 180 s tail.
3. **Draining at the per-peer limit.**
   - Re-sends run at 1384–1415 per second. That is the policy's limit: one per peer per 50 ms timer tick, or 9 members × 8 peers × 20 ticks = 1440 per second.
   - Unacked relays are 34 thousand (f = 2) and 30 thousand (f = 4) at 75 s, falling at about 1.4 thousand per second.
   - The 60 s tail is too short to see it finish. Round 2's 180 s tails show it recover.

What round 1 shows.

- **Controls.** Every control is stable at every f tried (note 1). The load alone is never too much.
- **Per-entry collapses** after the burst at every f from 1.5 to 8. Round 1 found no load level at which it recovers.
- **The two limited policies do not collapse anywhere.**
  - Per-entry with limit 3 recovers at f of 3 or more, and settles into the bounded 3-sends state at f = 2.
  - Per-peer drains at its limit.
- Rounds 2 and 3 repeat these with more seeds, and look for per-entry's boundary in the size of the burst.
- **No INVALID runs.** No node was killed for memory or exited.

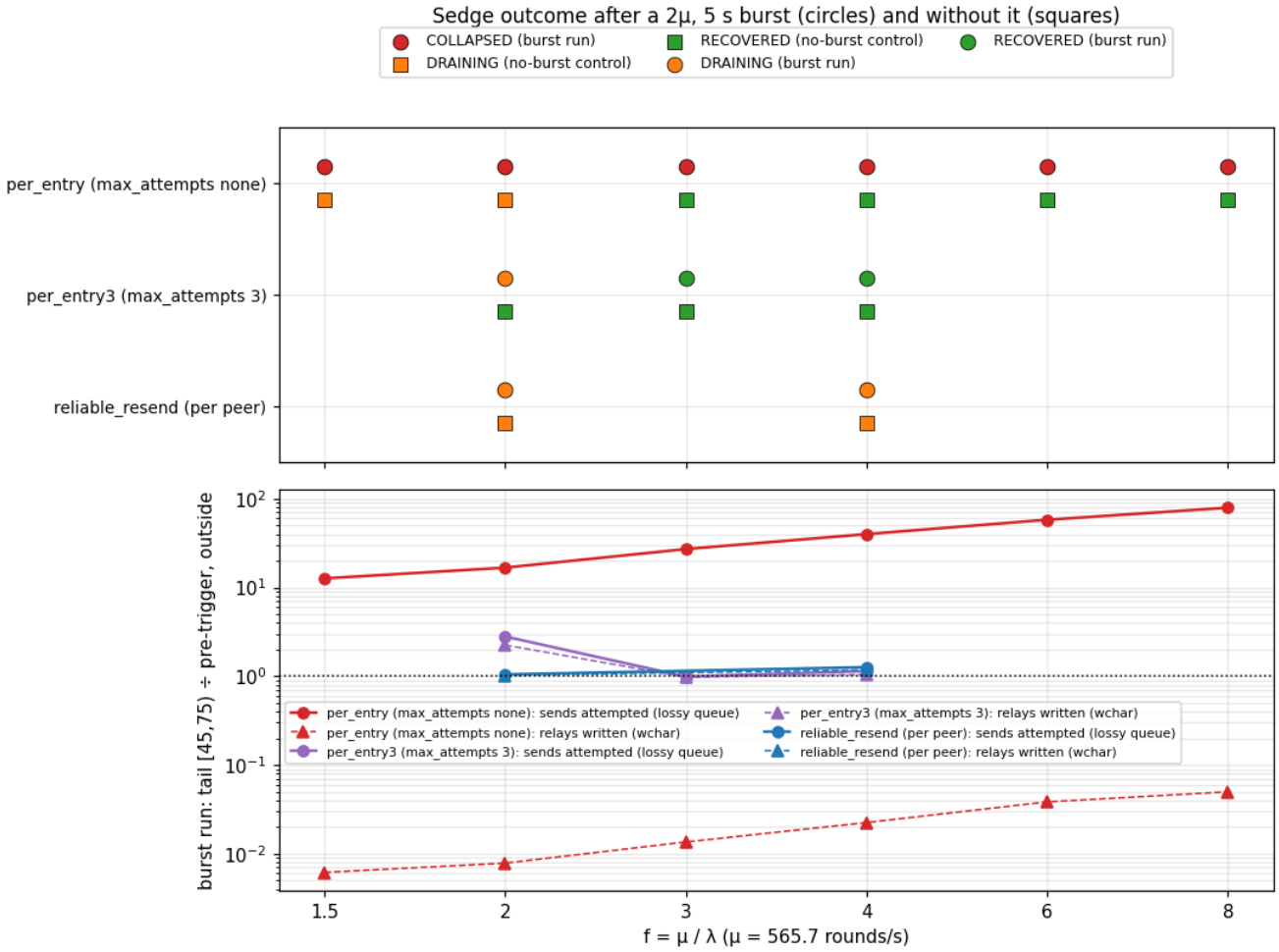


Figure 4: Outcome against f, and the tail read from outside

Is the collapse kept going by the extra work?

The question is not only whether something collapsed. It is whether the collapsed tail is spent on the outputs Cress flagged. For each run, the tail is compared with the same run's baseline and with its control.[19] Each table below has, for the burst run and then the control, the baseline rate, the tail rate, and their ratio.

Per-entry, f = 4, collapsed. No member report reached Sedge in the tail (0 of 9 per second), so the in-band rows are dark.

stream	burst: baseline /s	burst: tail /s	burst: ratio	control: baseline /s	control: tail /s	control: ratio
re-sends	5.2	–	dark	–	–	–
wire (in band)	9659.6	–	dark	9660.8	10094.1	1.04×
merged = delivered (in band)	10861.6	–	dark	10865.2	11355.3	1.05×
acks sent (derived)	9635.8	–	dark	9639.4	10101.9	1.05×
wire, outside (kernel writes)	10029.9	225.2	0.02×	10023.1	10175.0	1.02×
sends attempted, outside (send queue)	9651.2	385724.5	39.97×	9767.8	10074.3	1.03×
sends dropped, outside (send queue)	0.0	385478.9	–	0.0	0.0	–

Per-entry, limit 3, f = 2, bounded. All 9 reports arrive, so the in-band rows are live.

stream	burst: baseline /s	burst: tail /s	burst: ratio	control: baseline /s	control: tail /s	control: ratio
re-sends	33.8	38489.9	1139×	–	–	–
wire (in band)	21313.8	58819.5	2.76×	20809.6	20135.5	0.97×
merged = delivered (in band)	23965.0	50156.2	2.09×	23414.6	22650.8	0.97×
acks sent (derived)	21355.0	47611.9	2.23×	20804.6	20134.1	0.97×
wire, outside (kernel writes)	21245.4	47813.4	2.25×	21360.7	20095.3	0.94×
sends attempted, outside (send queue)	20964.1	58782.1	2.80×	20660.9	20226.7	0.98×
sends dropped, outside (send queue)	0.0	11489.9	–	0.0	0.0	–

Per-peer, f = 4, draining, for contrast:

stream	burst: baseline /s	burst: tail /s	burst: ratio	control: baseline /s	control: tail /s	control: ratio
re-sends	0.4	1414.9	3537×	–	–	–
wire (in band)	9674.0	11603.1	1.20×	9659.2	10057.1	1.04×
merged = delivered (in band)	10898.8	12877.6	1.18×	10864.6	11313.9	1.04×

[19] The table printed by `plot_amplification.py`. In band, "wire" is the program's own count of relays sent; "acks sent" is derived from it. Outside, "kernel writes" is `wchar` divided by 4120 bytes, and the send-queue rows are `sendq_attempted` and `sendq_dropped`. The controls' empty re-send column is `resent_n`.

stream	burst: baseline /s	burst: tail /s	burst: ratio	control: baseline /s	control: tail /s	control: ratio
acks sent (derived)	9673.0	11602.3	1.20×	9638.8	10060.5	1.04×
wire, outside (kernel writes)	9949.4	11681.3	1.17×	9941.5	10115.0	1.02×
sends attempted, outside (send queue)	9184.2	11607.6	1.26×	9825.5	10068.4	1.02×
sends dropped, outside (send queue)	0.0	0.0	–	0.0	0.0	–

The controls report no re-sends at all, shown as “–”.

The criteria set before the runs, one by one, for the collapsed per-entry runs:

- *Re-sends stay far above baseline.* **Yes, read from outside.**
  - The members’ own count is dark.
  - But they hand their send queues 350–395 thousand relay-sized sends per second. That is 13–79 times baseline, where baseline is the 72 first relays per offered request.
  - In bytes, almost every attempted send is relay-sized (attempted kilobytes ÷ 4120 bytes ≈ attempted count). So these are relays, not acks or reports.
  - After the burst, the only relays a member sends, apart from first relays, are re-sends.
- *Traffic measured outside stays far above the 72 first relays per request.* **No.**
  - Kernel writes fall to 0.6–5 % of baseline.
  - The re-sends are produced, but 99.9 % are dropped at a full queue before they reach the kernel.
  - The queues are full because the peers they write to are also at the cap, and so read slowly.
- *Merged relays and acks sent rise with it.* **Cannot be seen.** Both are counted only in the members’ own reports, which are dark. The kernel’s read counter does not count the members’ socket reads.
- *Member CPU stays at the cap and memory below the limit.* **Yes.**
  - CPU is 0.99–1.00 of the cap, with 99 % of periods throttled, at every f.
  - That includes f = 8, where the same members use 39 % of the cap without the burst.
  - Memory settles near 95 MB, a quarter of the limit, and creeps by about 0.5 MB/s.
  - No member was killed for memory.
- *The controls at the same f are stable.* **Yes** (round 1, note 1).
- *The reports stop, and only Sedge’s outside reading says “collapsed”.* **The reports do stop.**
  - For f of 6 or less, the program’s own stale reading was RECOVERED.
  - Sedge’s COLLAPSED comes from its outside reading: “went silent in band; … its capped nodes wrote 1–7 % of their baseline bytes per second through the tail, at 99–100 % of their CPU quota”.
  - The verdict does not rest on one measurement, though. The outside send-queue counters show what the CPU is spent on.

The policy comparison is the strongest evidence. The three configurations share the deployment, the cap, the queues and the burst. Only the retry policy differs. And only the policy whose re-sends grow with the backlog collapses.

- **Per-entry** re-sends every overdue relay on every timer tick.
  - Once the burst fills the queues, acks are dropped with everything else, so no relay is ever acknowledged.
  - The whole backlog stays overdue and is re-sent every three timer ticks, using the whole cap.
  - The loop keeps itself going. Cress’s verdict for it is “grows”, 175.60×.
- **Per-entry, limit 3** gives a relay up after three sends, so its re-sends are at most 2 times the first relays.

  - It settles at 2.80 times baseline attempted sends and 2.25 times kernel writes, with 19.5 % of sends dropped. That is inside the 2.80× ceiling Cress reports for it (“growth stops”).
  - This is the one round-1 run in which the extra work shows on the wire.
  - It is degraded but not collapsed: the reports keep arriving and every request is delivered.
  - The cost is correctness. Under loss it gives relays up, which is the hole HydroFrac found.
- **Per-peer** re-sends at most one relay per peer per timer tick, 1440 per second across the cluster. That is about 7 % of baseline traffic at f = 2, and the backlog drains.

Time series

Per-entry, f = 4, collapsed. The first plot shows re-sends and unacked relays, as the members report them.

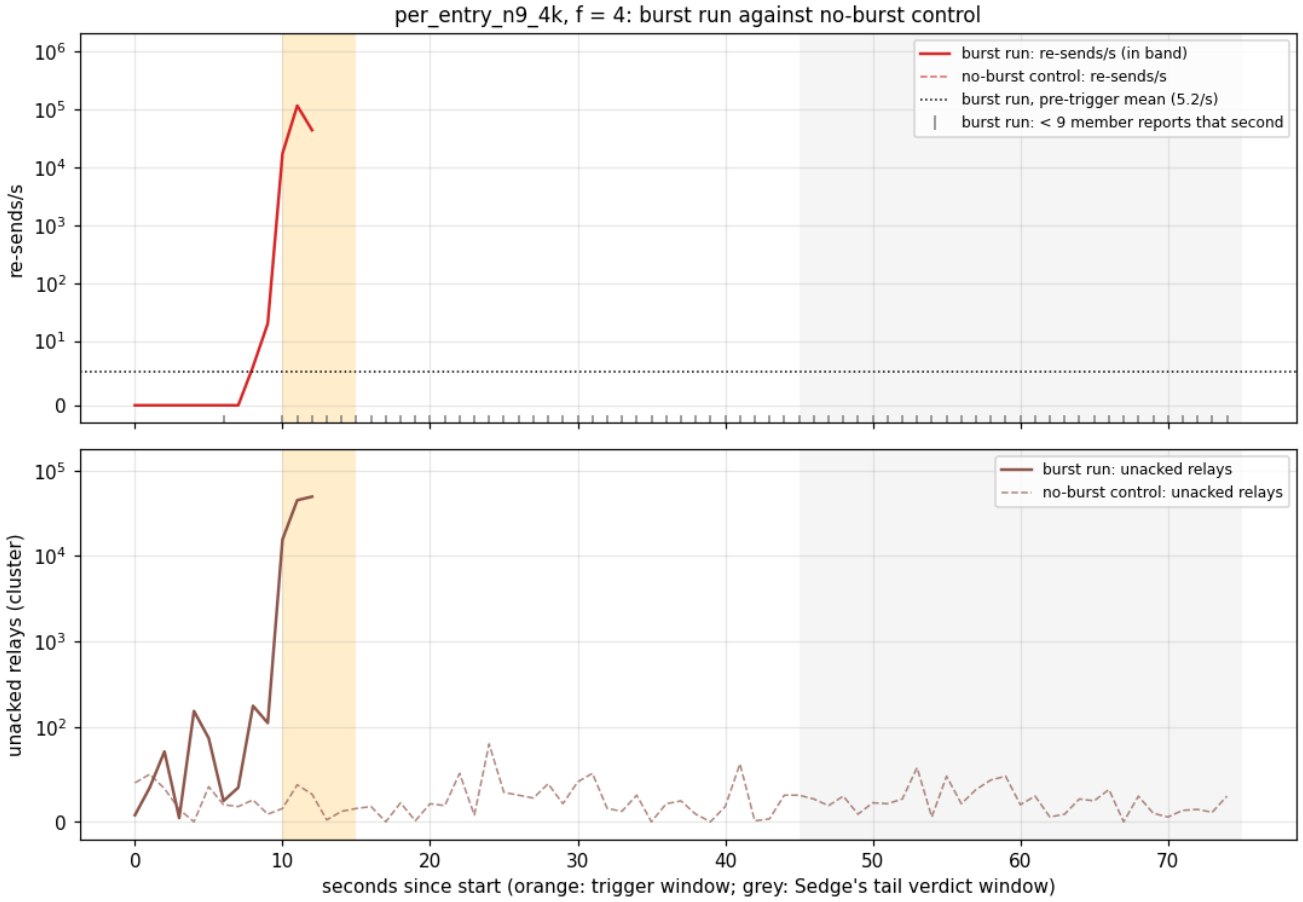


Figure 5: re-sends and unacked relays, per-entry f = 4

The reported series stop about 2 s into the burst. After that, no member report reaches Sedge.

The top panel is the core of the argument.

- Attempted sends (red) jump to about 380 thousand per second during the burst and stay there.
- Nearly all of them are dropped (orange).
- Kernel writes (blue) fall a hundredfold below the first relays the load needs (dotted).
- CPU stays at the cap, and memory stays at a quarter of the limit.

Per-entry, limit 3, f = 2: the bounded 3-sends state.

Per-entry, limit 3, at f = 4, and per-peer at f = 4, for contrast.

The remaining figures are in Appendix C.

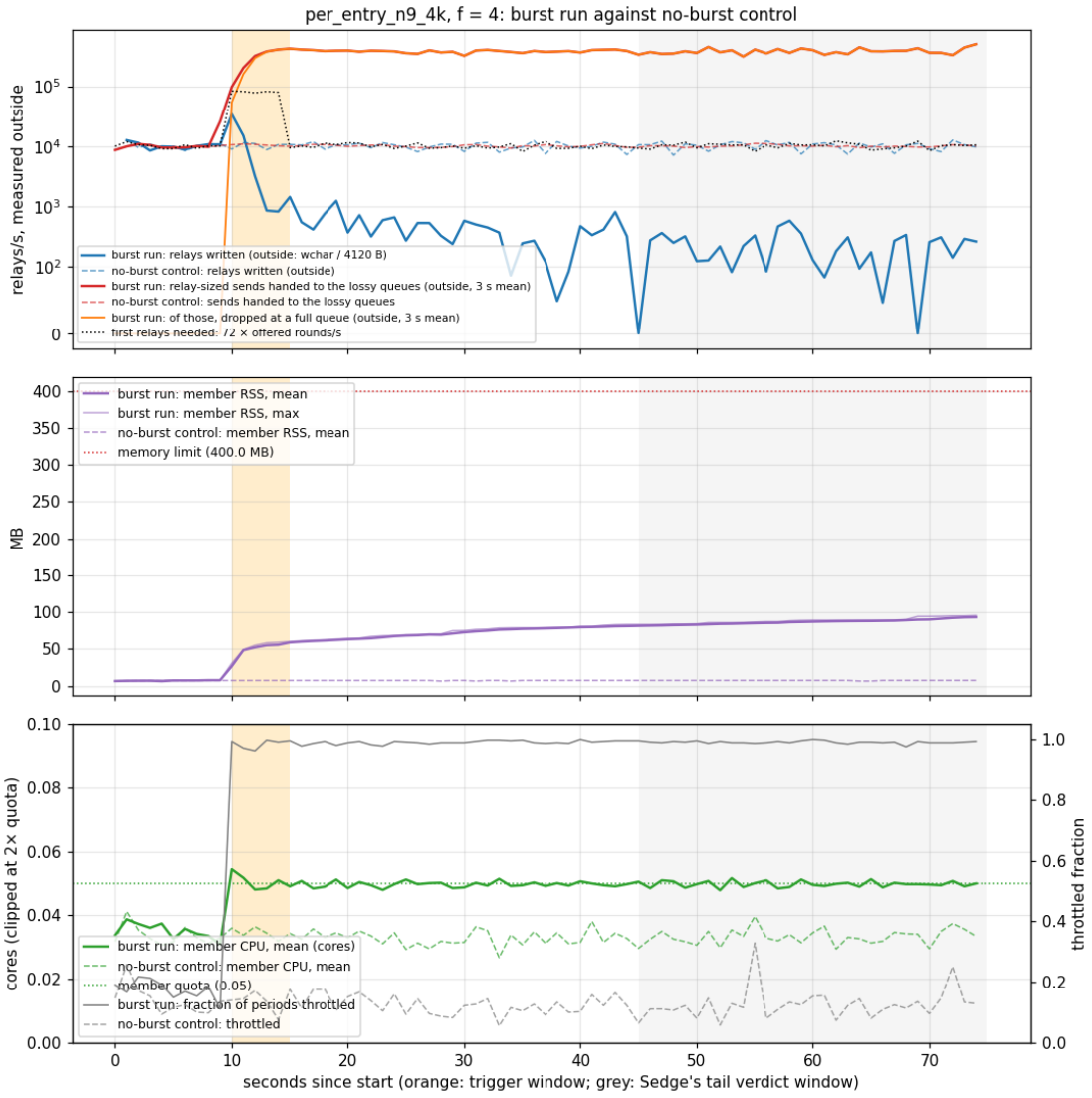


Figure 6: wire, attempted and dropped sends measured outside, memory and CPU, per-entry f = 4



Figure 7: tail against baseline, per-entry f = 4

CPU headroom at baseline

- A member's cost per request falls as load rises, because each step handles a larger batch. So under a fixed cap, at heavy load it uses nearly all of the cap.
- Under the old cap (0.01 of a core per 100 ms, unbounded queues), the average fraction of throttled periods at baseline was 0.98–1.0 at f = 1.5–2, 0.80–0.94 at f = 3, and 0.60–0.85 at f = 4.
- The new cap (0.05 of a core per 20 ms) gives real headroom from f = 3 on. From the per-entry controls, from 30 to 75 s:[20]

f	λ	member CPU / cap	fraction of periods throttled	average pause
1.5	377.1	1.00	0.86	13.5 ms
2	282.9	0.97	0.60	10.0 ms
3	188.6	0.82	0.25	7.8 ms
4	141.4	0.66	0.12	7.2 ms
6	94.3	0.51	0.07	7.2 ms
8	70.7	0.39	0.03	7.5 ms

- At f of 2 or less, the members are still near the cap at baseline. This cap did not give them real headroom there.
- The collapse does not depend on that. At f = 8 a member uses 39 % of its cap without the burst, and after the burst it is pinned at 100 %.

5. Discussion

The previous collapse was not caused by retries.

- The previous version of this demo found the per-peer policy collapsing at f of 2 or less after a burst. Its own analysis already saw the re-sends die down while the backlog kept growing.
- The cause was how the program kept its state, not its retry policy. The unacked list was rebuilt and filtered in full on every step. So every step cost work in proportion to the backlog, and the members' outgoing messages starved behind it.

[20] The tail window of `summarize.py`.

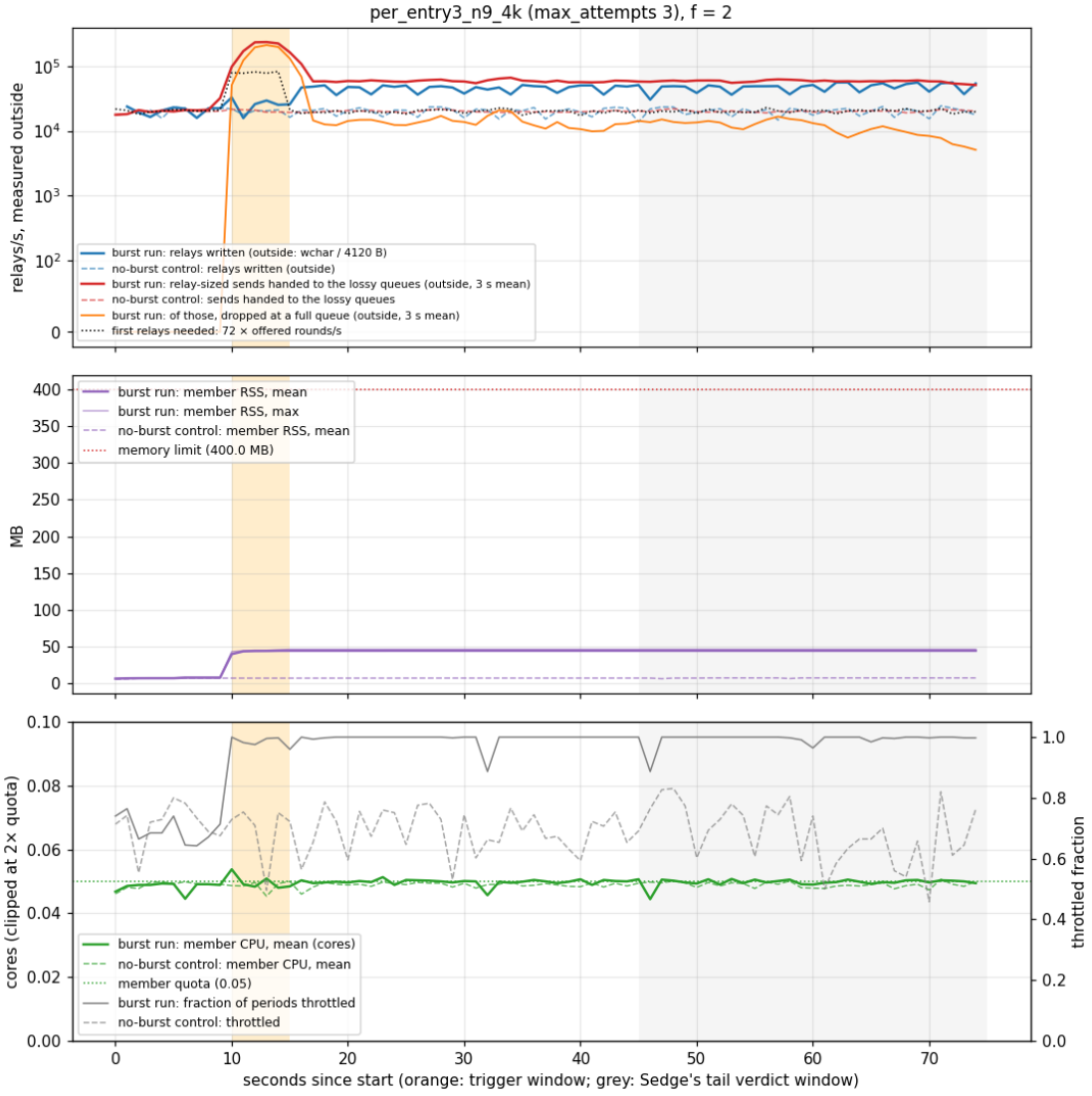


Figure 8: wire, attempted and dropped sends, per-entry limit 3, f = 2

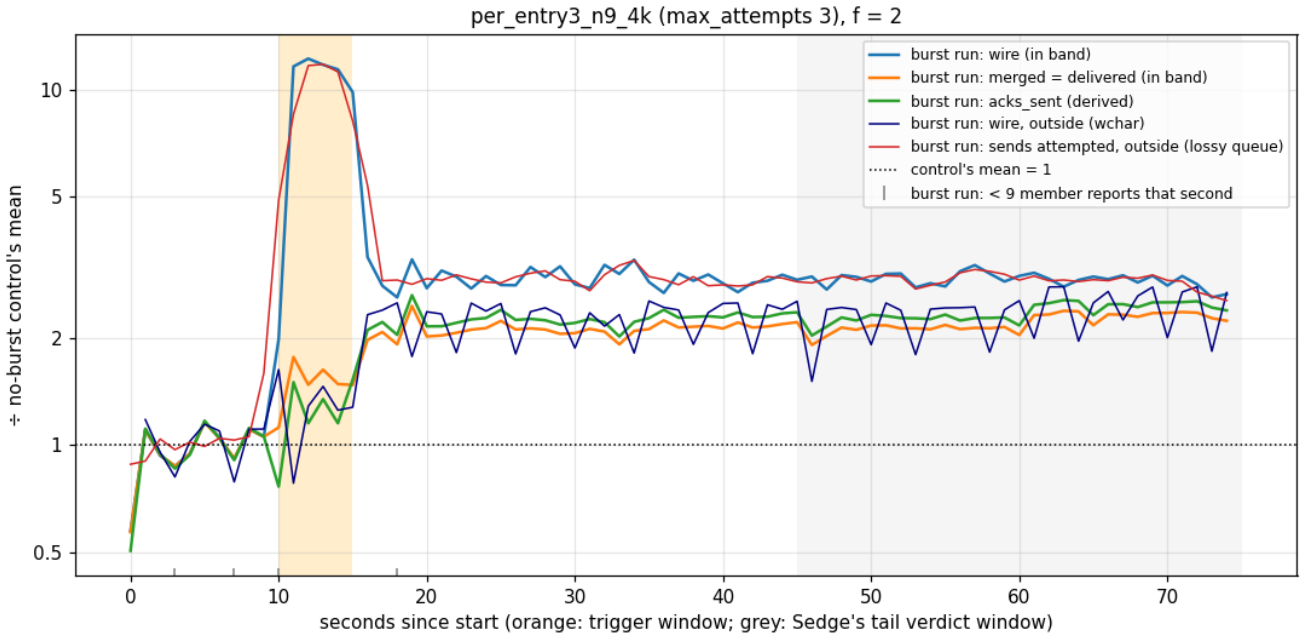


Figure 9: Cress's flagged outputs compared with the control, per-entry limit 3, f = 2

- With the list kept instead (old cap, unbounded queues):[21]
  - the same configuration and burst recover at every f tried (1.25, 1.5, 2 and 4);
  - they also recover from a burst four times as long, which builds a backlog of 250 thousand entries;
  - re-sends, limited to one per peer per timer tick, add about 270 per second across the cluster. That is about 4 % of normal traffic, far below the headroom.
- Cress counts messages, not the cost of each step, so it could not have seen this.
- Under this demo's deployment the per-peer policy still does not collapse. It drains at its limit (§4).

Re-sends that grow with the backlog, with unbounded queues: a crash.

- After a burst, with unbounded send queues, per-entry fell into a re-send storm of 6–10 thousand per second, at every f from 1.5 to 4.
- But the storm piled up in the program's send queues instead of going out on the wire. Members were killed at 400 MB, 30–45 s after the burst.
- Those runs are crashes, not collapses, and are reported as INVALID.
- Bounded queues that push back deadlocked the members instead.

With lossy queues: a collapse that holds without a crash.

- The storm is no longer stored. It is produced and thrown away.
- Member memory settles near 95 MB, no node dies, and the collapsed state holds through the whole 60 s tail at every f.
- The storm is still not on the wire in bytes: kernel writes fall to 0.6–5 % of baseline.
- What grows is the work of producing re-sends: 350–395 thousand relay-sized sends per second across the cluster, about 40 times baseline at f = 4. This uses the whole CPU cap.
- The queues drop them because every peer is busy doing the same thing.

The boundary is in the burst, not in the load.

- Round 1 found no f at which a burst of twice capacity for 5 s leaves per-entry unstuck, from f = 1.5 to 8. Round 2 repeats that at f = 2, 4 and 8 (9 of 9).

[21] `relay_broadcast_amplification/README.md`, Part 1.

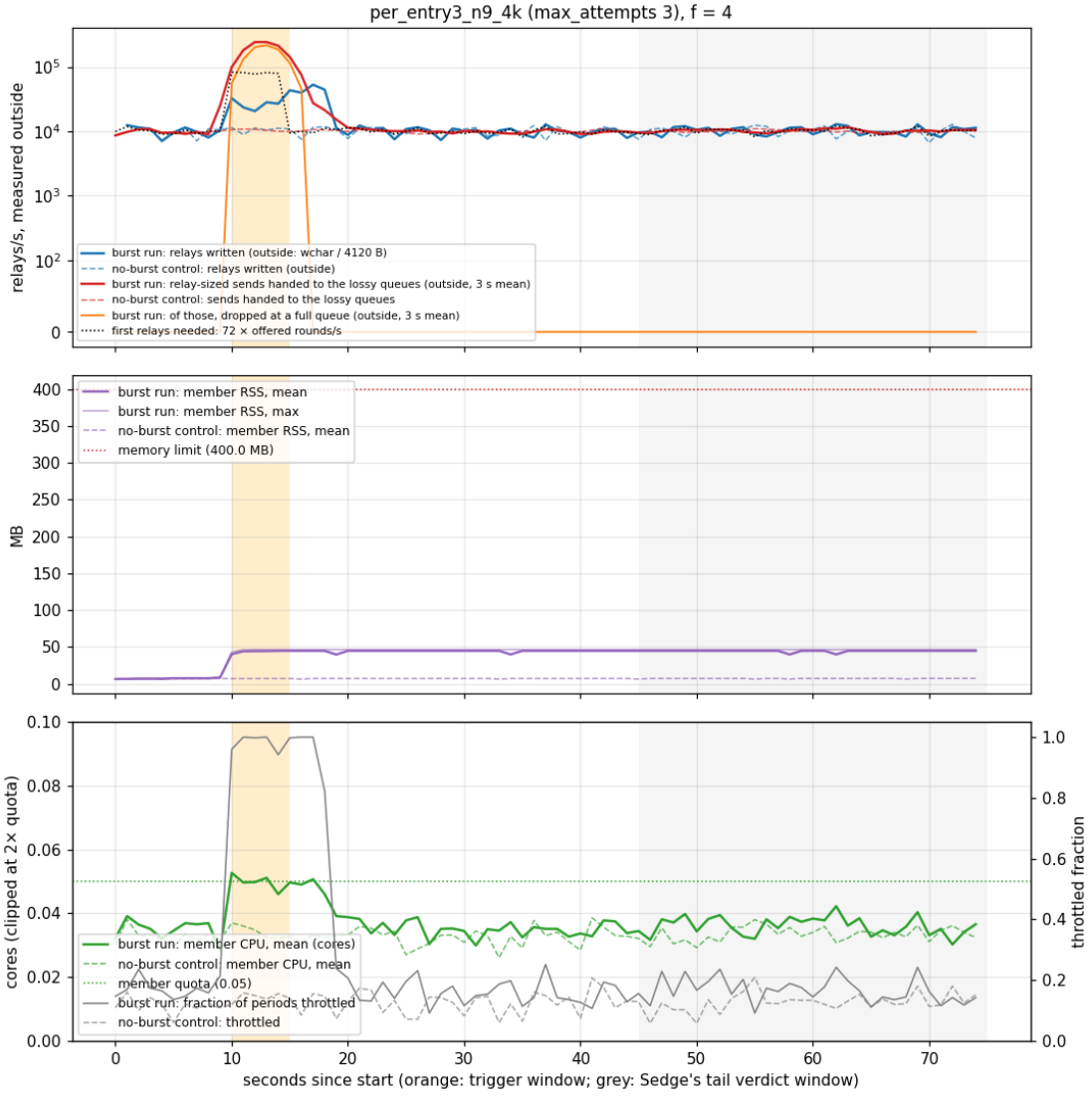


Figure 10: wire, attempted and dropped sends, per-entry limit 3, f = 4

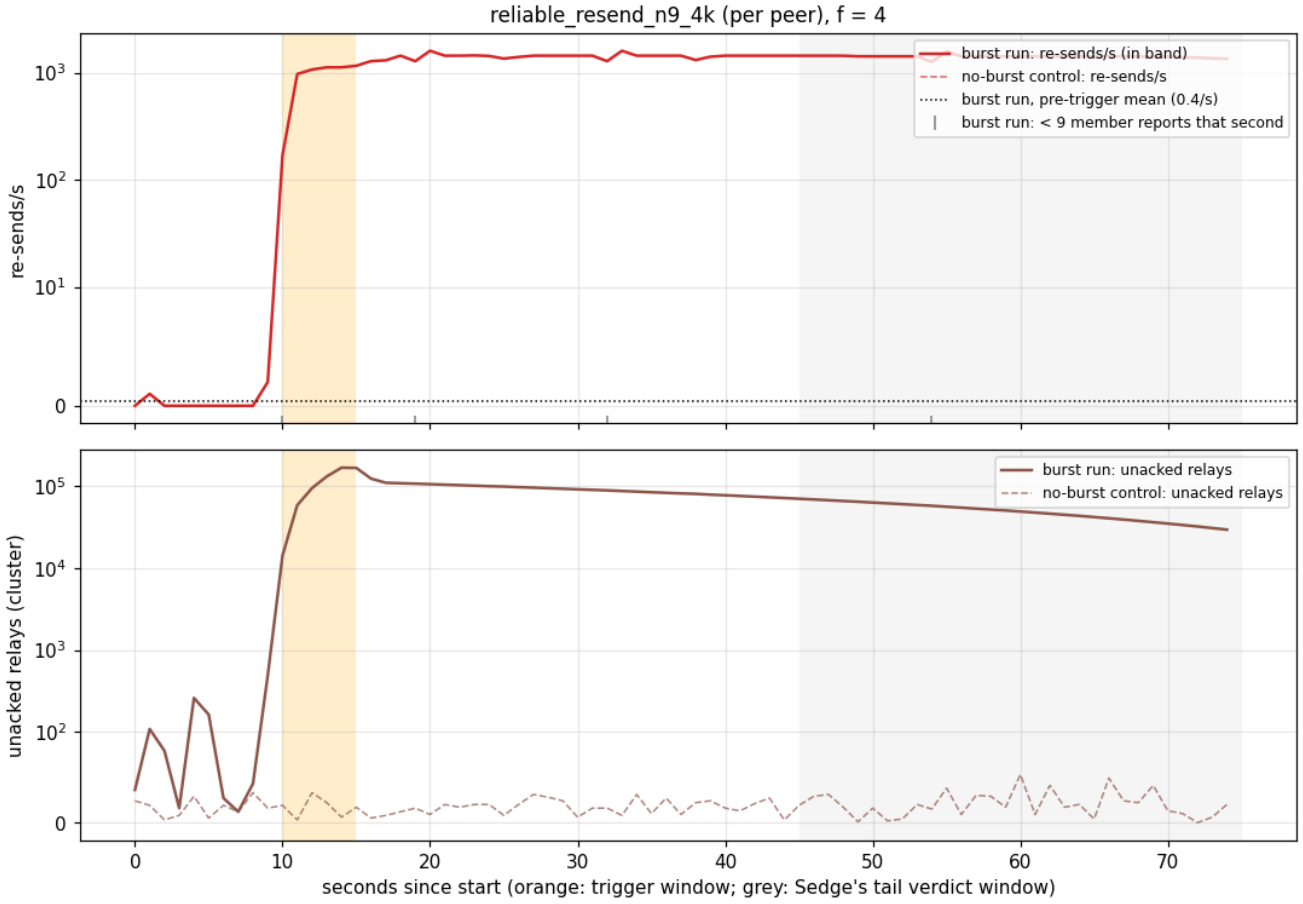


Figure 11: re-sends and unacked relays, per-peer f = 4

- On isolated cores at f = 4, the burst has to be made small before any run escapes: 4 of 5 stuck at 1.5μ for 1 s, and 3 of 5 still stuck after 180 s at 2μ for 0.5 s.
- The same small burst produces both endings. Seed 32 storms on the wire for 100 s and drains. Seed 34 storms for 150 s and falls into the black hole.

The limited variant storms but does not get stuck.

- Per-entry with limit 3 sends each relay at most three times.
- Its storm always drained: it never got stuck within 180 s, and no run reached a black hole.
- How long the storm lasts depends on $3\lambda/\mu$, the re-send load at the limit relative to capacity. It lasts a few seconds below 1, 18–39 s at 1.35, and 16 s to more than 60 s at 1.5.
- Given the uncertainty in μ, the point where the storm starts to outlast the burst lies between $3\lambda/\mu \approx 1.0$ and 1.35.

Conclusion. The extra work Cress identifies is what the members spend their capacity on after the burst, and it keeps the collapse going.

- In the storm, the extra re-sends are on the wire: about 3 times the kernel writes, for as long as 150 s.
- In the black hole, the same loop runs, but almost all of its output is dropped at full queues. The acks that would end it are lost with everything else, so it never ends within the run.
- Changing only the retry policy changes the outcome. With growth that stops (per-entry, limit 3), the storm ends. With the per-peer limit, the system recovers (round 2, 180 s tails).

What this evidence does **not** show is extra bytes on the wire *in the black hole*. There, kernel writes fall to a few percent of baseline. The extra work is visible only in the transport layer's attempted-send counters and in CPU pinned at the cap.

Caveats

- **One machine.** Round 1 ran each setting once, round 2 at most 3 times, and round 3 5 times. Rounds 1 and 2 did not isolate cores that share hardware. Each round-3 boundary rests on 5 seeds on one core set.
- **Capacity is uncertain.** Three measurements on one core set gave 654–778 requests per second. That is about ±9 % around their middle, and the probe's own step of about 9 % leaves at least ±5 %. From lowest to highest, the spread is about 19 %. So every boundary given in units of μ is a range.
- **Kernel writes fall in the stuck state.** In the per-entry collapse the members write 0.6–5 % of their baseline bytes. The extra sends are counted by the transport layer's lossy queue, inside the member process but outside the program's own logic. The kernel does not count them.
- **The members' reports go dark** in every per-entry collapse. So Cress's "merged" and "acks sent" outputs cannot be measured there.
- **The source's backlog grows.**
  - The collapsed members read slowly, so the source's sends to them queue up in the uncapped source.
  - In round 1 its memory reaches 722 MB at f = 1.5 by 75 s (limit 2000 MB).
  - It hands the members only 18–49 % of the offered requests at f of 4 or less.[22]
  - Its memory keeps growing while a black hole lasts. In the longest one (seed 31, 180 s tail) it reached 552 MB.
  - No node died in any of the final runs.
- **Per-entry with limit 3, at f = 2, storms for longer than a 60 s tail.** In five 180 s tails it ended within 16–66 s.
- **Cress's verdict** was computed on a reduced workload with empty bodies, and Cress models delay, not loss (§3).
- **HydroFrac's "no violation found"** covers sampled orderings at 3 members and 2 ids.
- **No "necessary but not sufficient" claim.** An earlier draft said the extra work was necessary but not sufficient for the collapse. That claim was dropped. This document claims only what is

[22] The `intake` column of `summarize.py`.

listed under the Conclusion.

Appendix A: Methods

Isolated cores (round 3)

The machine has 8 physical cores with 2 hardware threads each. Logical CPUs n and n + 8 share one physical core: its execution units and caches. Round 3 runs in two *core sets*:

- Each core set is one CPU for the test harness plus two CPUs for the program.
- The CPUs that share hardware with them (9–11 and 13–15) are left idle.
- CPU 0 and its partner, CPU 8, are used only for builds, Cress and analysis.[23]

core set	harness CPU	program CPUs	μ from three measurements (requests/s)	μ used
S1	1	2–3	654.3, 654.3, 713.5	654.3
S2	5	6–7	713.5, 654.3, 778.1	713.5

- **Round 2 had no such isolation.** Its core sets shared physical cores with each other's load, so a node's capacity depended on what its partner thread was running. Its boundary numbers are therefore not used. Only its large-margin results are: a burst of twice capacity for 5 s, and 180 s or 300 s tails.
- **Round 1 also predates the isolation.** Its program ran on CPUs 7–8, which share hardware with CPUs 15 and 0. Its outcomes have large margins, but treat its numbers near a boundary with that in mind.

Measuring capacity

- The probe steps through rates about 9 % apart.
- Three measurements on the same core set gave 654.3–778.1.[24]
- Pairs of cores whose partner threads were busy gave 462–733.[25]
- Near capacity, a probe can itself set off per-entry's re-send storm, so its result there is not repeatable.
- So every ratio to μ in this document is given over that range, and f is computed from the μ the core set used.

The silent-tail rule

The members' reports travel the same queues as their traffic. So in a black hole they stop reaching Sedge, and the program's own values stay stuck at their last reading. The build that ran round 3 applies one rule to every run:

- If more than 20 % of the tail's seconds have no member report at all, the tail is judged from outside.
- Then, if the capped nodes' kernel writes over the second half of the tail are at most 50 % of their baseline rate, the outcome is **COLLAPSED**.
- Otherwise it is **INVALID**: the program cannot be read, and its writes are not low enough to call it collapsed.

Applying the rule to every run changed 4 earlier runs from INVALID to COLLAPSED, all from round 2, so all large-margin. Their outside writes, as a fraction of baseline, over the whole tail and then over the second half of the tail:[26]

[23] `relay_broadcast_lossy/README.md`.

[24] `calibration_smt_isolated/`.

[25] `calibration_index.tsv` and `calibration_per_slot/`.

[26] `relay_broadcast_amplification/rescore_silent.py`. The groups are `r2_A_pe_f4_m1.5_d1.5_s5` (trials 1, 2, 3) and `r2_A_pe_f4_m2_d0.5_t180_s5` (trial 2).

- burst of 1.5µ for 1.5 s, runs 1, 2 and 3: 0.51 → 0.03, 0.52 → 0.03, 0.76 → 0.04;
- burst of 2µ for 0.5 s with a 180 s tail, run 2 (seed 13): 0.63 → 0.01.

Over the whole tail, the storm before the black hole lifts the average above the 50 % threshold. The second half sees only the black hole.

Two round-3 runs stay INVALID under the rule. Both are still storming on the wire. Their reports are missing for 13 and 49 tail seconds, and their writes are at about 60 % and 100 % of baseline.

One outside-only classification for the fraction tables

Sedge's labels mix the members' reports with outside readings. Its DRAINING label is often noise in the slope of a flat line (round 1, note 1). So the fraction tables also classify every run, controls included, the same way, from outside counters over the second half of the tail:[27]

- **black hole:** kernel writes below 0.2 times the baseline (5–10 s), with member CPU above 0.9 of the cap;
- **storm:** sends handed to the lossy queues above 1.5 times baseline;
- **clear:** otherwise.

"Stuck" means black hole or storm. All 50 round-3 controls are clear.

The lossy transport

- The lossy send queue is an option of the deployment layer, not of the program.[28]
- Each outgoing connection gets a queue of at most 1024 messages and its own writer.
- A send that finds the queue full is dropped.
- The transport counts every send it is handed and every drop, in messages and in bytes, and publishes the counts for Sedge.
- These counters live inside the member process, but outside the program's own logic. They keep counting when the program's reports are lost.

The counter-recording fix

- A member's own counters (re-sends, wire, and so on) arrive as one increment per report.
- Sedge used to record an increment of n as n separate records.
- When a starved member's report finally arrived carrying millions of re-sends, that loop stalled the harness for hundreds of milliseconds. The same harness task also paces the open loop feed.
- That could make the feed late, and the run INVALID, in exactly the runs that matter.
- Sedge now keeps an increment as one entry that stands for n records.[29]

Appendix B: Commands

All commands run from `design_docs/sedge_outside_capacity/`, except the `cargo`, `./scripts/cress` and `./scripts/sedge` commands, which run from the repository root.

Sedge and Cress runs. Cress first, on every new configuration, then the burst runs, the controls, and the summary script.

```
# Cress first, on every new configuration.
./scripts/cress --config per_entry_n9_4k hydro_test relay_broadcast
./scripts/cress --config per_entry3_n9_4k hydro_test relay_broadcast
./scripts/cress --config reliable_resend_n9_4k hydro_test relay_broadcast
# Burst runs (C = per_entry_n9_4k, per_entry3_n9_4k or reliable_resend_n9_4k).
./scripts/sedge --config C --deploy-profile profile \
  --cpus Member=0.05 --cpu-period-us 20000 --send-buffer lossy:1024:Member \
  --memory-mb Member=400 --memory-mb Source=2000 --mu 565.7 \
```

[27] `plot_outcomes.py`.

[28] `--send-buffer lossy:1024:Member`, a hydro_deploy option.

[29] `sedge/src/trace.rs`, test `a_large_counter_report_is_one_entry`.

```
  --harness-cpus 5-6 --node-cpus 7-8 --factors F --trials 1 --tail 60 \
  hydro_test relay_broadcast
# The no-burst control at factor F.
./scripts/sedge ... --trigger-multiple $(python3 -c "print(1/F)") ...
# One line per run: the verdict and the tail against the baseline, in band and outside.
python3 relay_broadcast_amplification/summarize.py --quota=0.05 RUN_DIR ...
```

- Round 1 used seed 1 for every run.
- Round 3 used core set S1 (`--harness-cpus 1 --node-cpus 2-3 --mu 654.3`) and S2 (`--harness-cpus 5 --node-cpus 6-7 --mu 713.5`).
- Round 3 ran 5 runs per setting: seeds 21–25 with a 60 s tail, and 31–35 with a 180 s tail.
- The per-entry burst sizes were set with `--trigger-multiple M --trigger D`.
- Every group's exact arguments and core set are in relay_broadcast_lossy/commands.txt.

Plots. The plot scripts read only committed traces. The block below rebuilds every figure in this document.

```
python3 -m pip install --target $TMPDIR/py matplotlib markdown-it-py "fpdf2<2.8"   # Python 3.7
export PYTHONPATH=$TMPDIR/py
R=relay_broadcast_lossy/runs
tr() { ls $R/$1/*.factor$2.trial$3.tsv; }   # group, factor, trial -> trace
# Round 1: one call per burst run, each with the no-burst control of the same factor and length.
for p in "pe 4" "pe3 2" "pe3 4" "rr 4"; do set -- $p
  python3 demo_relay_broadcast/plot_amplification.py \
    --burst $(tr p_$1_burst_f$2 $2 0) --control $(tr p_$1_ctl_f$2 $2 0) \
    --out demo_relay_broadcast --prefix $1_f$2 --memory-mb 400 --quota 0.05 --period-us 20000
done
python3 demo_relay_broadcast/plot_summary.py $R demo_relay_broadcast/lossy_summary.png p_
# The headline: seed 34 (stuck) and seed 32 (drained), each with its matched control.
for p in "3 34" "1 32"; do set -- $p
  python3 demo_relay_broadcast/plot_headline.py \
    --burst $(tr r3_S2_pe_f4_m2_d0.5_t180_s5 4 $1) --control $(tr r3_S2_pe_f4_ctl_d0.5_t180_s5 4 $1) \
    --out demo_relay_broadcast/headline_pe_f4_seed$2.png
done
# Round-3 outcome fractions (also prints the two tables).
python3 demo_relay_broadcast/plot_outcomes.py $R demo_relay_broadcast/r3_outcomes.png
```

Appendix C: Where the data is

Everything here was produced on the branch `feat/hydrofrac-cress-sedge-e2e`.

Machine. The Sedge runs used one machine: 16 logical CPUs (Intel Xeon Platinum 8375C at 2.90 GHz), cgroup v1, Docker 25.0.14. Round 1 ran the harness on CPUs 5–6 and the program on CPUs 7–8. Round 3 used two isolated core sets (Appendix A).

Raw data. All of it is under `relay_broadcast_lossy/`. Its README.md describes every file.

- `runs/<group>/`: Sedge's `results.tsv` row and per-second trace for every run, with the outside columns: `wchar_kb_*` (kernel writes), `rss_mb_*` (memory), `cpu_*`, `nr_throttled_*`, `throttled_ms_*`, and `sendq_attempted[_kb]_*`, `sendq_dropped[_kb]_*` (send-queue counters). There are 106 groups.
  - Round 1: `p_pe_*` (`per_entry_n9_4k`), `p_pe3_*` (`per_entry3_n9_4k`) and `p_rr_*` (`reliable_resend_n9_4k`), each as `burst_fF` and `ctl_fF`.
  - Round 2: `r2_*`. Only its large-margin groups, `r2_C_rr_*` and `r2_D_pe_*`, are used here.
  - Round 3: `r3_S1_pe3_f<F>[_t180]_s5[_ctl]` and `r3_S2_pe_f4_{m<M>,ctl}_d<D>[_t180]_s5`, where the burst is M·μ for D seconds.
- commands.txt: the exact Sedge arguments and core set of every group.
- `probes.tsv`, `calibration.tsv`, `calibration_per_slot/`, `calibration_smt_isolated/`, calibration_index.tsv and `probes_index.tsv`: the capacity probes.
- `cress/`: the three Cress reports, and `console.txt` with their verdict lines.
- `earlier/`: the unbounded and pushing-back (`bounded:Member`) runs that ruled out those deployments.

- `demo_relay_broadcast/hydrofrac.txt`: the HydroFrac verdicts (16 tests, re-run on commit `52597a61df`).
- `relay_broadcast_amplification/`: the earlier per-peer and unbounded-queue study that §5 refers to, with the scripts `summarize.py` and `analyze2.py`.

Figures. Made by the scripts in `demo_relay_broadcast/`. Besides the figures shown above, that directory has `*_streams.png`, `*_tail.png` and `*_resends.png` for every round-1 pair.